

1. TITLE

AUTOMATED PODCAST TRANSCRIPTION, TOPIC SEGMENTATION & SENTIMENT ANALYSIS SYSTEM FOR AUDIO CONTENT INTELLIGENCE

2. OVERVIEW

This project presents an automated system for analyzing long-form podcast recordings by generating timestamp-aligned transcript segments, concise summaries, and emotional sentiment tracking. The system processes raw audio data to clean, transcribe, and segment text based on semantic topic shifts. It further extracts key discussion themes, visualizes sentiment trends over time, and produces structured analytical outputs. The final output is presented via an interactive dashboard, enabling efficient retrieval and analysis of podcast content while significantly reducing manual listening effort.

3. APPROACH

The application is designed to assist researchers, content creators, and listeners in efficiently reviewing long podcast episodes. Instead of listening to entire audio files, users receive timestamped transcript segments, extracted keywords, sentiment graphs, and abstractive summaries. The modular design allows easy extension and integration into other systems such as media monitoring dashboards or content archives.

Core Methodology

This section explains the end-to-end logic implemented in the project modules.

Step 1: Audio Preprocessing

- **Module:** podcast_backend.py (preprocess_audio)
- Raw audio files (MP3/M4A) are loaded and converted to WAV format.
- Audio is resampled to **16,000 Hz** and mixed to **Mono** to meet the input requirements of the Whisper model.

Step 2: Speech-to-Text and Timestamp Extraction

- **Module:** podcast_backend.py (transcribe_and_summarize)
- Processed audio is fed into the **OpenAI Whisper** model.
- Text is generated along with precise start and end timestamps for every sentence.
- Data is saved as structured JSON for downstream analysis.

Step 3: Semantic Topic Segmentation

- **Module:** podcast_backend.py (segment_topics)
- The transcript is split into sentences and vectorized using **TF-IDF**.

- **Cosine Similarity** is calculated between sliding windows of text.
- When similarity drops below a threshold (0.65), a "Topic Shift" is detected, creating a new segment with its own timestamp range.

Step 4: Sentiment Analysis

- **Module:** podcast_backend.py (analyze_sentiment)
- **NLTK VADER** analyzes each sentence to calculate a compound emotion score (-1 to +1).
- This data generates the "Emotional Journey" timeline graph in the dashboard.

Step 5: Keyword Extraction

- **Module:** podcast_backend.py (extract_keywords)
- TF-IDF analysis identifies words that are statistically unique to the specific episode compared to general language.
- Top ranking terms are saved as "Tags" for the episode.

Step 6: Summary Generation

- **Module:** podcast_backend.py (transcribe_and_summarize)
- The full text (or specific segments) is fed into the **DistilBART** Transformer model.
- An abstractive summary is generated to provide a quick overview of the content.

Step 7: Output Visualization

- **Module:** dashboard.py
- Streamlit reads the generated JSON/TXT files.
- It renders interactive charts (Plotly) and displays the transcript with search highlighting.

Testing and Quality Assurance

To ensure system reliability and code stability, a comprehensive Unit Testing framework was implemented using Python's unittest library. This testing suite validates critical backend components in isolation before full pipeline execution.

Key Testing Areas:

- **Utility Logic:** Verifies data formatting functions, such as converting raw seconds into human-readable timestamps (MM:SS), ensuring accurate time display in the dashboard.

- **NLP Functionality:** Tests the keyword extraction algorithms with sample text inputs to confirm valid output generation (strings/lists) without running heavy models.
- **System Integrity:** Validates the automated directory setup process, ensuring that all necessary storage folders (audio, transcripts, summaries) are created correctly upon initialization.

Outcome: The test suite allows for automated health checks of the codebase (python -m unittest discover tests), catching errors early and ensuring that core logic functions correctly before processing large audio files.

4. TECH STACK

Core

- **Python 3.9+**
- **OpenAI Whisper:** State-of-the-art ASR model.
- **Librosa / Soundfile:** Audio loading, resampling, and normalization.
- **FFmpeg:** System-level audio processing engine.

NLP & AI

- **Hugging Face Transformers:** sshleifer/distilbart-cnn-12-6 for summarization.
- **Scikit-Learn:** TF-IDF Vectorizer, Cosine Similarity.
- **NLTK:** VADER for sentiment, Punkt for sentence tokenization.

Visualization & UI

- **Streamlit:** Interactive web application framework.
- **Plotly Express:** Interactive charts (Sentiment Timeline, Pie Charts).

Storage

- **JSON:** Structured storage for transcripts, segments, and analysis data.
- **TXT:** Human-readable summaries and logs.

System Architecture Flow

Audio Input (Upload or URL)



Audio Preprocessing (Resampling to 16kHz Mono)



Speech-to-Text & Timestamp Extraction (OpenAI Whisper)



Semantic Segmentation & Topic Detection (TF-IDF + Cosine Similarity)



Sentiment Analysis & Keyword Extraction (VADER + TF-IDF)



Summarization (DistilBART Transformer)



Visualization & UI (Streamlit Dashboard)

5. SETUP

Software Requirements

- **Python 3.9+**
- **Windows, Linux, or macOS** operating system
- **FFmpeg** (System-level audio tool)

Project Directory Structure

project/

```
|
|
| └─ data/          # Data Storage
|
|   └─ audio/       # Raw Input Audio
|
|   └─ processed_audio/  # 16kHz WAVs
|
|   └─ transcripts/  # JSON Transcripts with timestamps
|
|   └─ semantic_segments/  # Topic Segmentation Reports
|
|   └─ sentiment_data/  # Sentiment scores for graphing
|
|   └─ short_summary/  # AI Summaries
|
|   └─ keywords/     # Extracted Keywords
|
|
| └─ src/           # Source Code
```

```
| ├── app.py          # User Interface (Streamlit)
| ├── podcast_backend.py  # Core Logic Engine
| └── __init__.py
|
| ├── docs/          # Documentation files
| ├── requirements.txt    # Project Dependencies
| └── README.md        # Quick Start Guide
```

Dependency Installation

pip install -r requirements.txt

6. USAGE

1. **Launch the App:** Run `streamlit run src/dashboard.py` in your terminal.
2. **Input Data:** Use the Sidebar to upload an MP3 file or paste a podcast URL.
3. **Process:** Click "Start Analysis" to trigger the AI pipeline.
4. **Explore:**
 - View the **Executive Summary** and **Keywords**.
 - Analyze the **Sentiment Graph** to see emotional trends.
 - Browse **Topic Segments** to jump to specific parts of the conversation.
 - Search the **Full Transcript** for specific terms.

Results:

- **High-Accuracy Transcription:** Robust performance on diverse audio inputs.
- **Intelligent Segmentation:** Successfully breaks long audio into 4-8 distinct, coherent chapters.
- **Visual Analytics:** Provides clear insights into the emotional tone and content distribution of the episode.
- **Efficiency:** Reduces the time required to understand a 1-hour podcast to just a few minutes of reading.

7. TROUBLESHOOTING

- **FFmpeg Error:** If you see an error related to audio loading, ensure FFmpeg is installed and added to your system's PATH variable.

- **Memory Issues:** Running Whisper and BERT models simultaneously can be memory-intensive. Ensure you have at least 8GB of RAM available.
- **Download Timeouts:** Large files or slow internet can cause timeouts during the initial model download (Whisper/BART). Run the script once with a stable connection to cache the models locally.

8. LIMITATIONS AND CHALLENGES

- **Processing Time:** Analyzing long audio files on a standard CPU can take significant time (approx. 0.5x real-time). GPU acceleration is recommended for faster performance.
- **Speaker Diarization:** The current system does not distinguish between different speakers (Speaker A vs. Speaker B), which can make multi-person podcasts harder to follow.
- **Language Support:** While Whisper supports multiple languages, the current NLP pipeline (Sentiment/Keywords) is optimized primarily for English.

9. FUTURE SCOPE

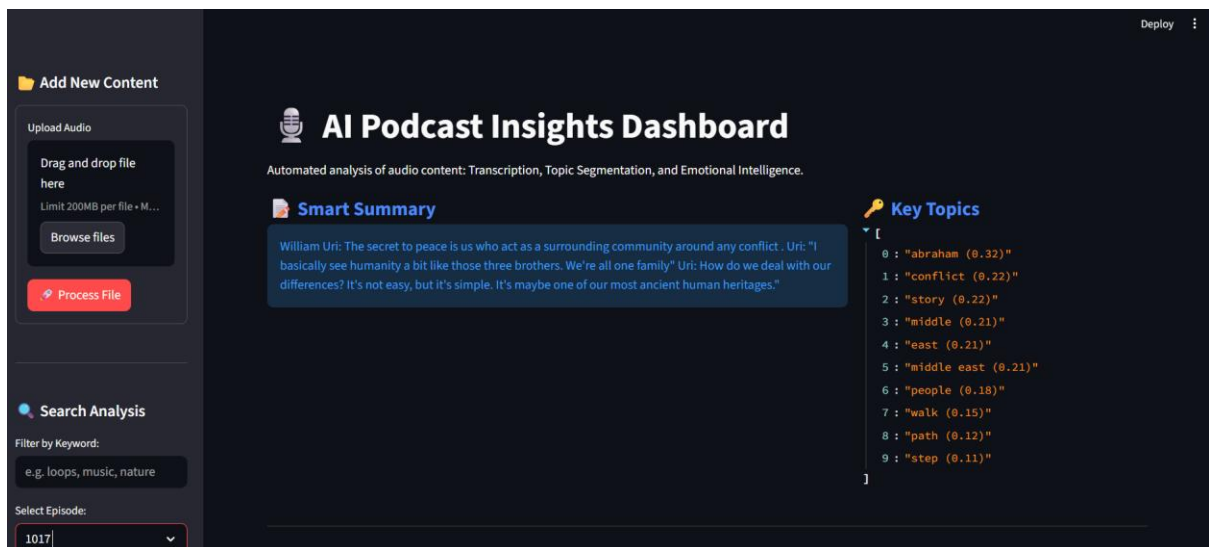
- **Speaker Identification:** Integrate Pyannote.audio to label different speakers in the transcript.
- **Real-time Processing:** Optimize the pipeline to handle live audio streams.
- **Multi-language NLP:** Expand the sentiment and keyword extraction modules to support other languages natively.
- **Cloud Deployment:** Deploy the application to a cloud platform (AWS/GCP/Streamlit Cloud) for wider accessibility.

10. REFERENCES

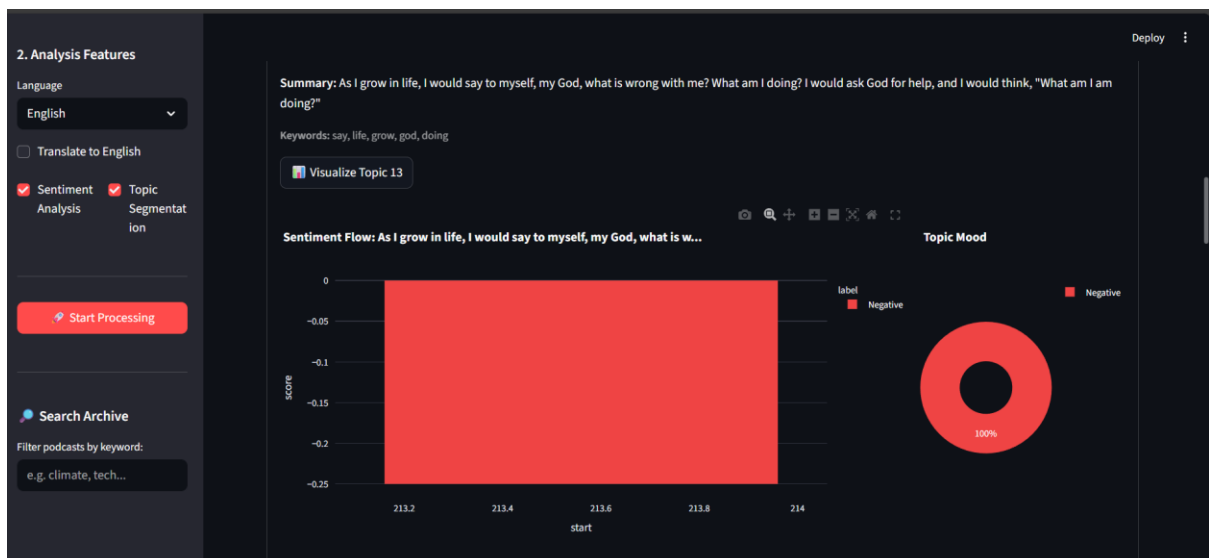
- **OpenAI Whisper:** <https://github.com/openai/whisper>
- **Hugging Face Transformers:** <https://huggingface.co/docs/transformers/index>
- **Streamlit Documentation:** <https://docs.streamlit.io/>
- **NLTK Library:** <https://www.nltk.org/>

11. SCREENSHOTS

Dashboard Overview



Sentiment Analysis Graph



Topic Segmentation View

